First Automation FIN1209: automate Chapter 8's window oscillators in R

Write a short R script that reads price files, finds where the RSI, the MACD and the stochastic give buy and sell signals, counts how many agree, and draws them. Then draw a flowchart that explains it.

Student worksheet 100 points | a few hours over a week

Benjamin C. Sotelo | Institute of Accounts, Business and Finance, FEU Manila

Name _____ Section _____ Date _____

What you are making

Read this page first. It is short.

You will teach R to read the **window oscillators** of Chapter 8. You put a Google Sheets price file in a folder, run a short **R script**, and it works out, day by day, where the **RSI**, the **MACD** and the **stochastic** each give a buy or sell signal, how many of them agree, and the price to act at. It writes a table and a chart. Add new prices and run it again, and both update. That is **automation**.

> YOU DO NOT NEED TO KNOW R
>
> Nobody in this class was taught to code. The script is under 50 lines, Part 3 shows what it does, and Part 5 builds it in seven small steps. The numbers it prints are not advice: Part 7 shows what they mean, and why signals that agree may agree for a bad reason.

What you get, and what you hand in

	WHAT
You get	This manual, a video that shows every step on screen, the finished example script and the example flowchart (`.pdf` and `.bpmn`). No price files: you download them yourself.
You hand in	Your own R script and a flowchart of how it works. Your own stock, and your own pair of Chapter 8 oscillators (Part 9).
Due date	_____ (to be filled in: after the Saturday 7 November Chapter 8 class)

> WHAT THESE SIGNALS ARE
>
> The output is **the signals of three rules, applied to past prices**. It is not investment advice and it does not say what price will do next. Chapter 1's point still holds: a forecast cannot be checked when it is made.

1. What to install

Free, and about 30 minutes. If you cannot install, use the browser option.

You need two programs. **R** is the language, and it does the work. **RStudio Desktop** is a friendly window for writing and running R. Install R first, then RStudio.

> NO R PACKAGES
> The example needs **no packages at all**. It uses only what comes with R. You never type `install.packages`. Your own script should not need one either, because the marker's computer will not have it.

W Windows

1. Go to **https://cran.r-project.org/bin/windows/base/** and click the **Download R** link at the top. Open the file you downloaded and click Next through the installer, keeping every default.
2. Go to **https://docs.posit.co/ide/user/download.html** and click the big **Download for Windows** button. Open the file and click Next through the installer.
3. Open **RStudio** from the Start menu. A window opens with a pane called Console on the left. That is where R talks.

M Mac

1. Go to **https://cran.r-project.org/bin/macosx/**. Click the first `.pkg` under the latest release, the one for your chip (the Apple icon, then About This Mac, tells you whether it is Apple silicon or Intel). Open the file and click Continue through the installer.
2. Go to **https://docs.posit.co/ide/user/download.html**, click **Download for macOS**, open the file, and drag RStudio into the Applications folder.
3. Open **RStudio** from Applications. A window opens with a pane called Console on the left.

C If you cannot install anything

Use **Posit Cloud**, which runs RStudio in your browser. Go to **https://posit.cloud**, make a free account, then **New Project**, then **New RStudio Project**. Everything in this handout works the same way. To put the example folder in, use the **Upload** button in the Files pane.

F For the flowchart

Draw it in **bpmn.io** at **https://demo.bpmn.io/**. It runs in the browser, is free, and needs no account. Or install **Camunda Modeler** from **https://camunda.com/download/modeler/**. Both draw BPMN and both save a `.bpmn` file and export a picture. If you would rather use Markdown, **https://mermaid.live** draws a Mermaid flowchart in the browser.

2. Where to get price data, and what kind

Checked on the public sites on 4 November 2026. Where a claim needs an account, it says so instead of guessing.

Chapter 1 names the data technical analysis reads: **price, volume and open interest**. For this activity you need one bar per trading day, with six columns.

The script needs six columns: Date, Open, High, Low, Close and Volume. The RSI and the MACD use the close; the stochastic uses the high, low and close; a buy's entry price uses the high. Volume is not used here (Chapter 6 covers it). **Open interest** exists only for futures, so a stock has none, and no source below gives one for a share.

The free sources, and what we found

SOURCE	OPEN, HIGH, LOW, CLOSE, VOLUME	ACCOUNT	CSV FILE
Google Sheets, GOOGLEFINANCE	Yes, daily or weekly, all five, with the formula `"all"`. The Help page lists them.	A Google account	Yes: File, Download, CSV. **This handout uses it, for US-listed stocks only.** `PSE:JFC` returned `#N/A`, so there is no daily history for Philippine stocks.
PSE EDGE (edge.pse.com.ph), the exchange's own site	Today's open, high, low, last price and volume, and the 52-week range, for each Philippine stock. We found no table of past days.	None to read	No download link found. The site says its content is copyrighted.
Yahoo Finance	A history table of date, open, high, low, close, adjusted close and volume for US stocks, readable without signing in. We found no Philippine stocks.	None to read the table	No Download button when signed out. Whether one appears after signing in is not checked.
Investing.com	Not checked: the site refused our automated visit. It is a common place to look for Philippine stocks, so check what it offers for the one you want.	Not checked	Not checked
TradingView	Charts Philippine stocks, for example PSE:BDO, with a free public page. Export of the chart data was not checked.	None to view; export needs an account	Not checked, needs an account
Your broker's platform, for example COL Financial	Not checked: it sits behind a login.	Yes, a brokerage account	Not checked

Prices in the **adjusted close** column are corrected for splits and dividends. The example does not use it.

The script works with any source whose CSV has the columns of Part 5. Google Sheets is the one route this handout walks through step by step, because every student already has an account and it exports a CSV in the right layout. If you find Philippine data elsewhere and can save it as a CSV with those column names, it will run.

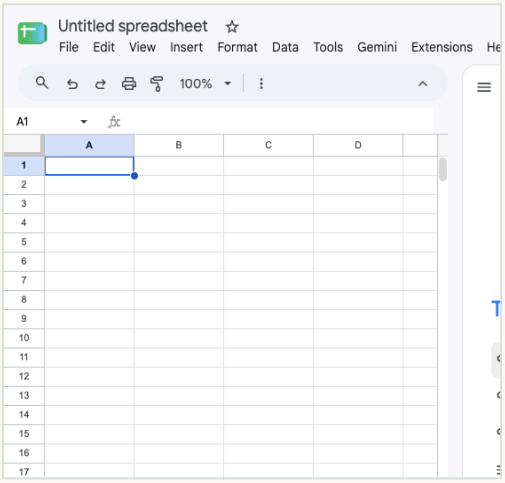
2b. Get the example prices yourself

The same route you will use for your own task. We give you no price files.

Google Sheets can fetch daily prices with one formula, and it gives exactly the columns the contract needs: Date, Open, High, Low, Close, Volume. It works for **US-listed stocks**. You do this once per stock, and you use the same dates as below so your numbers match this handout.

1 Open a blank spreadsheet

Go to **https://sheets.new** while signed in to your Google account. A blank spreadsheet opens. Click cell **A1**.



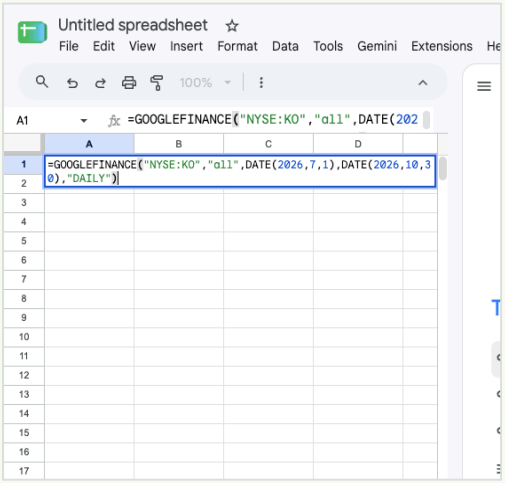
2 Type the formula

In A1 type this formula for Coca-Cola, then press Enter:

TYPE EXACTLY THIS

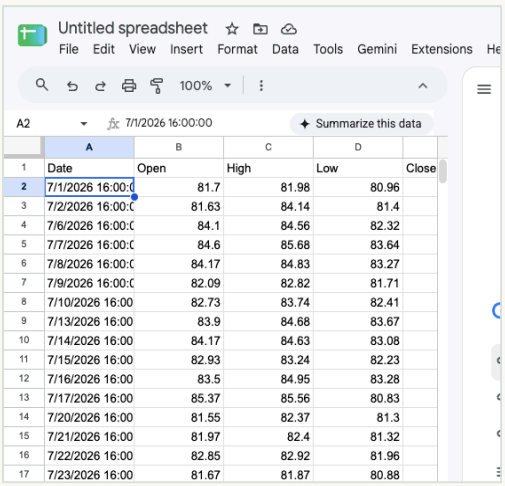
```
=GOOGLEFINANCE("NYSE:KO","all",DATE(2026,7,1),DATE(2026,9,30),"DAILY")
```

The picture shows the formula being typed. Its end date in that capture reads 2026,10,30; **use 2026,9,30** for this first run.



3 Check the result

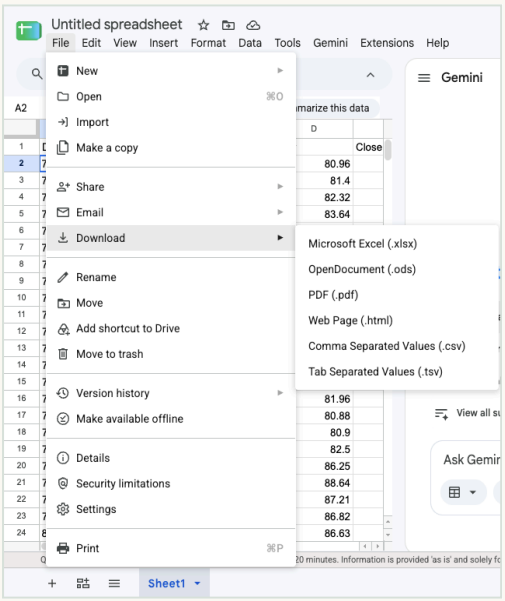
Wait a moment. About 64 rows fill down from A1, with the six column names in row 1. The dates read like `7/1/2026 16:00:00`: that is the layout Google Sheets gives, and the script reads it as it is, so you never edit a date. If you see `#N/A`, check the quote marks are straight and the ticker is spelled `NYSE:KO`.



	Date	Open	High	Low	Close
2	7/1/2026 16:00:0	81.7	81.98	80.96	
3	7/2/2026 16:00:0	81.63	84.14	81.4	
4	7/6/2026 16:00:0	84.1	84.56	82.32	
5	7/7/2026 16:00:0	84.6	85.68	83.64	
6	7/8/2026 16:00:0	84.17	84.83	83.27	
7	7/9/2026 16:00:0	82.09	82.82	81.71	
8	7/10/2026 16:00	82.73	83.74	82.41	
9	7/13/2026 16:00	83.9	84.68	83.67	
10	7/14/2026 16:00	84.17	84.63	83.08	
11	7/15/2026 16:00	82.93	83.24	82.23	
12	7/16/2026 16:00	83.5	84.95	83.28	
13	7/17/2026 16:00	85.37	85.56	80.83	
14	7/20/2026 16:00	81.55	82.37	81.3	
15	7/21/2026 16:00	81.97	82.4	81.32	
16	7/22/2026 16:00	82.85	82.92	81.96	
17	7/23/2026 16:00	81.67	81.87	80.88	

4 Download it as a CSV

Choose **File, Download, Comma Separated Values (.csv)**. The file holds the values, not the formula, so there is nothing more to do to it. Your browser saves a file named after the spreadsheet.



Rename the file **KO.csv** and move it into your `data` folder. The name matters: the script takes the ticker from it. Then do the same for a second stock in a new spreadsheet, with this formula, and save it as **F.csv** in `data`. Part 6 runs the script on these two files.

CELL A1, FOR F (FORD)

```
=GOOGLEFINANCE("NYSE:F","all",DATE(2026,7,1),DATE(2026,9,30),"DAILY")
```

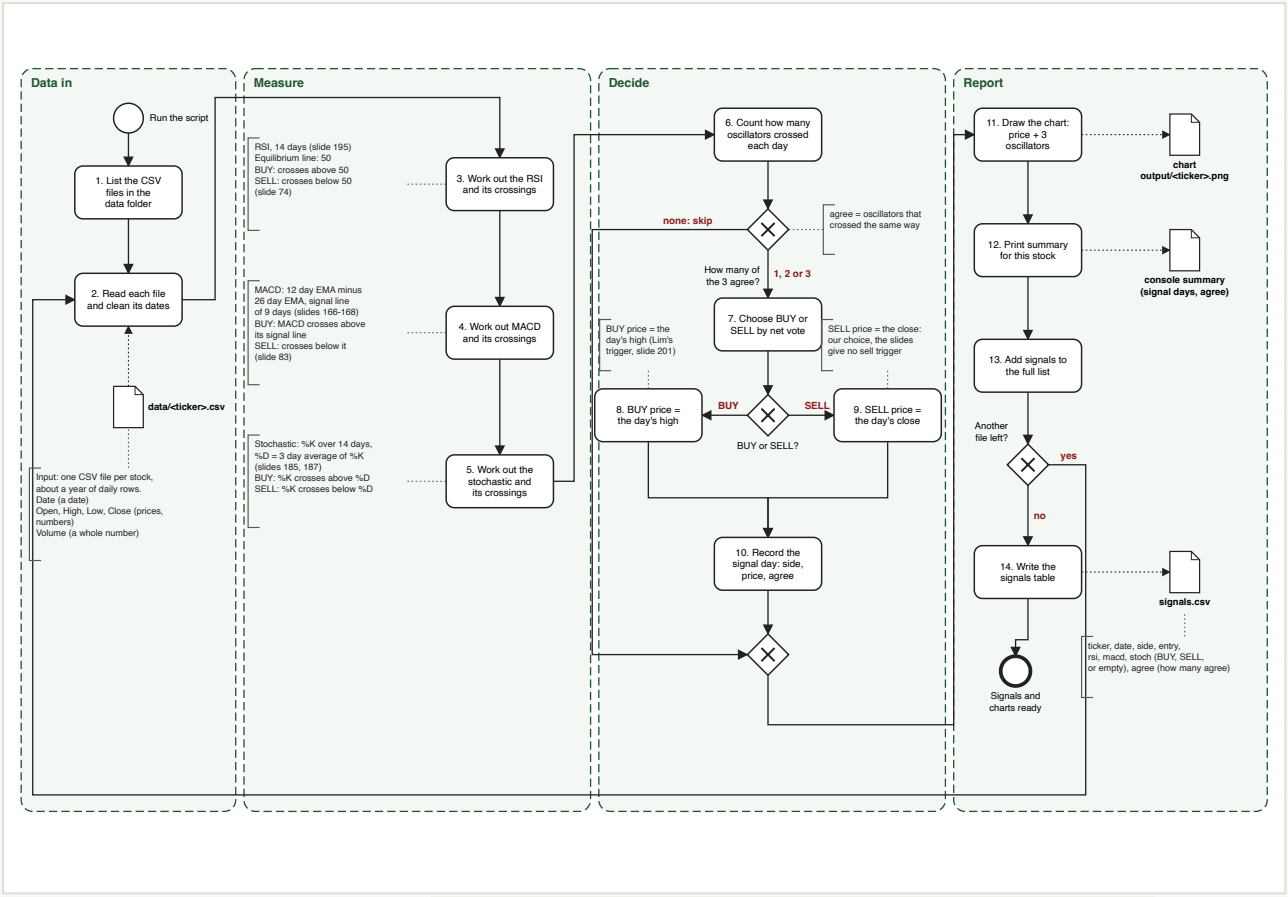
Later, for the update in Part 6: in each spreadsheet change `DATE(2026,9,30)` in the formula to `DATE(2026,10,30)`, download again, and replace the file in `data`, still named KO.csv and F.csv. There is no capture of that second download because it is the same four steps.

SWAP THE TICKER, NOTHING ELSE

To use another US stock, change only the text inside the first quotes of the formula, for example `"NASDAQ:AMD"`. **Philippine stocks do not work here.** We tried `PSE:JFC` and Sheets answered `#N/A`: GOOGLEFINANCE gives no daily history for Philippine stocks. For a PSE stock you need another source from Part 2, for example Investing.com, then save its data as a CSV with the columns of Part 5.

3. What the script does

Input, process, output. A flowchart says it in shapes; yours must match your script line for line.



The example script as a BPMN flowchart. `First Automation FIN1209 - Example Flowchart.bpmn` opens in bpmn.io.

Drawn for this course by build/bpmnkit.py.

Input, process, output

PART	WHAT HAPPENS
Input	A price file per stock in the `data` folder, from Google Sheets.
Process	For every stock and every day: work out the RSI, the MACD and the stochastic. A signal is the day a line crosses another (RSI over 50, MACD over its signal line, %K over %D). Count how many oscillators crossed the same way that day: that is `agree`.
Output	A table of the signal days with a price to act at, and a chart of price with the three oscillators beneath it.

Which lines each shape stands for

SHAPE	SAYS	LINES
Start event	Run the script	the moment you click Source
1	List the CSV files in the data folder	line 16
2	Read each file and clean its dates	lines 17 to 19
3	Work out the RSI and its crossings	line 20 and line 23
4	Work out the MACD and its crossings	line 21 and line 24
5	Work out the stochastic and its crossings	line 22 and line 25
6	Count how many oscillators crossed each day	line 26
Gateway	How many of the 3 agree?	line 27 and the agree column
7	Choose BUY or SELL by the net vote	line 28
8, 9	BUY price = high, SELL price = close	line 30
10	Record the signal day	lines 29 to 32
11	Draw the chart: price and three oscillators	lines 33 to 43
12	Print summary for this stock	line 45
13	Add signals to the full list	line 44
14	Write the signals table	line 47

BPMN needs five shapes: a thin circle starts, a thick circle ends, a rounded rectangle is a task named with a verb, a diamond with an X is a yes or no question with every arrow labelled, and an arrow is the order. Every shape on your flowchart should point at lines of your script.

4. The three rules

Each rule as the Chapter 8 slides give it. Slide numbers are those of the Chapter 8 student edition.

What the slides say

SLIDE	RULE
74	An oscillator's equilibrium is 50 for a bounded one. Crossing above it is a buy signal and crossing below it a sell signal. The RSI is bounded, so we use its 50 line.
83, 166, 168	A signal line is a moving average of the oscillator itself. The MACD is a 12 period EMA minus a 26 period EMA, with a 9 period signal line. Crossing above the signal line is a buy and crossing below it a sell.
185, 187, 189	The fast stochastic: %K is where the close sits in the last 14 bars' range, and %D is a 3 period average of it. %K crossing above %D is a buy, below it a sell.
193 to 195	The RSI uses 14 periods, with the average gain and average loss as plain averages (the book gives no formula for Wilder's own).
201	Lim says the trigger for entry is normally the high of the bar tied to the buy signal.
78, 81	Oscillators built from the same prices are highly correlated and give the same signals. Lim calls this multicollinearity and says it creates a false consensus.

OUR CHOICES, NOT LIM'S RULE

Entry price. A buy uses the signal day's high, as slide 201 says. The slides give no trigger for a sell, so a sell uses the day's close. **That is our choice, not Lim's rule.**

One vote each. A signal day is any day at least one oscillator crosses. If a buy and a sell cancel, the day is skipped. `agree` counts the oscillators that gave the winning side.

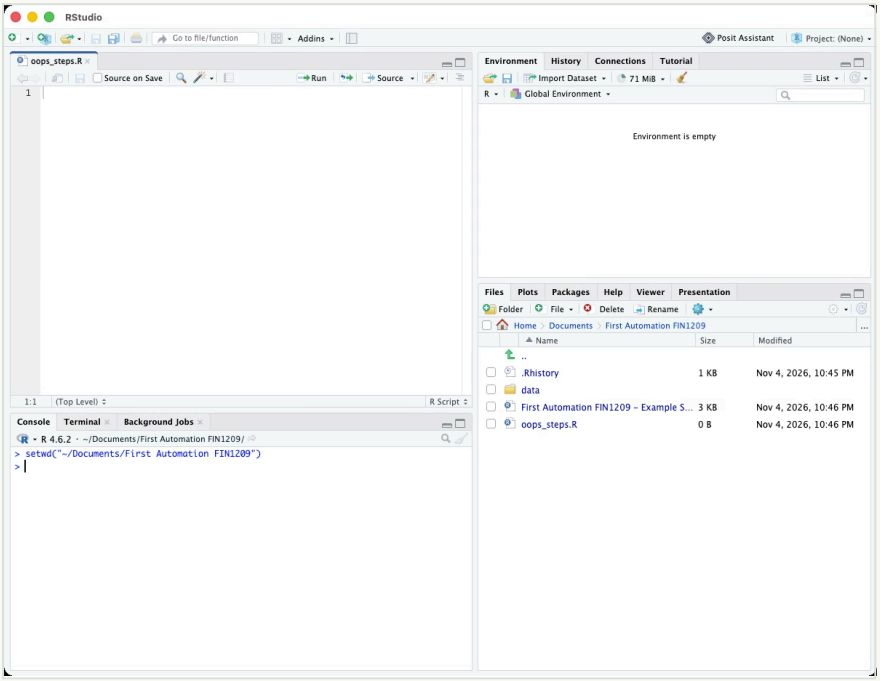
No trend filter, no exit. The slides give none that a computer can run, so the script reports the signals and stops.

5. Write the script in seven steps

Each step adds one idea. The new lines are highlighted, and what R printed is under them.

1 Set up

Make a folder `First Automation FIN1209` in your Documents folder with a folder `data` inside it, and put the two Part 2 files `KO.csv` and `F.csv` in `data`. In RStudio choose **File, New File, R Script**, save it in the folder, then **Session, Set Working Directory, To Source File Location**. Type `install.packages("TTR")` in the Console once. Run your script with **Code, Source**.



1 Read a price file

A **variable** is a name that holds a value: `ticker <- "KO"` puts the text KO into the name `ticker`. `read.csv()` opens a CSV file as a table, one row per trading day. `paste0()` joins text. Google Sheets writes dates like `7/1/2026 16:00:00`: `sub()` cuts the time off and `as.Date()` makes a real date. A line starting with `#` is a note for people.

NEW IN STEP 1

```
ticker <- "KO"
bars <- read.csv(paste0("data/", ticker, ".csv"))
bars$Date <- as.Date(sub(" .*", "", bars$Date), tryFormats = c("%Y-%m-%d", "%m/%d/%Y"))
print(head(bars, 3))
```

R PRINTED

```
        Date  Open  High   Low Close   Volume
1 2026-07-01 81.70 81.98 80.96 81.29 21734809
2 2026-07-02 81.63 84.14 81.40 84.14 18320542
3 2026-07-06 84.10 84.56 82.32 82.96 18992644
```

2 Use a package: the RSI

R's maths comes in **packages**. TTR holds the technical indicators. Install it once, with `install.packages("TTR")` typed in the Console, then `library(TTR)` loads it. `RSI()` returns the relative strength index, 14 days, with the plain averages the slides use (slide 195). `tail()` shows the last rows. The RSI swings between 0 and 100 around its equilibrium, 50 (slide 74).

NEW IN STEP 2

```
library(TTR)
rsi <- RSI(bars$Close, n = 14, maType = "SMA")
print(tail(rsi, 3))
```

R PRINTED

```
[1] 43.09133 45.60099 39.76608
```

3 A crossing is a signal

A **function** packs lines under a name. `cross(a, b)` gives +1 on the day line a crosses above line b and -1 on the day it crosses below: the RSI crossing 50 is a buy or sell signal on slide 74. `which()` finds the days that are not zero.

NEW IN STEP 3

```
cross <- function(a, b) {        # +1 on the day a crosses above b, -1 on the day it crosses below
  a <- round(a, 6)
  b <- round(rep(b, length.out = length(a)), 6)     # rounding stops a rounding-error tie from counting as a cross
  before <- function(x) c(NA, head(x, -1))
  s <- ifelse(a > b & before(a) <= before(b), 1, ifelse(a < b & before(a) >= before(b), -1, 0))
  s[is.na(s)] <- 0
  s
}
word <- function(s) ifelse(s == 1, "BUY", ifelse(s == -1, "SELL", ""))
rsi_sig <- cross(rsi, 50)
print(bars$Date[which(rsi_sig != 0)])
```

R PRINTED

```
[1] "2026-07-23" "2026-07-27" "2026-08-17" "2026-08-19" "2026-09-08"
[6] "2026-09-22" "2026-09-23"
```

4 Two more oscillators, and a vote

The same `cross()` reads the other two rules. The MACD is a 12 day average minus a 26 day average, with a 9 day signal line (slides 166, 168), and its crossing of the signal line is a signal (slide 83). The stochastic %K crossing %D is a signal on slide 187. Adding the three signals gives a vote: positive is a buy, negative a sell, and the size says how many agree.

NEW IN STEP 4

```
macd <- MACD(bars$Close, nFast = 12, nSlow = 26, nSig = 9, percent = FALSE)
sto <- stoch(bars[, c("High", "Low", "Close")], nFastK = 14, nFastD = 3)
macd_sig <- cross(macd[, "macd"], macd[, "signal"])
stoch_sig <- cross(sto[, "fastK"], sto[, "fastD"])
votes <- rsi_sig + macd_sig + stoch_sig
day <- which(votes != 0)
side <- ifelse(votes[day] > 0, "BUY", "SELL")
print(table(side))
```

R PRINTED

```
side
 BUY SELL
  12   15
```

5 Keep the signals in a table

Each signal day becomes a row: the date, buy or sell, the entry price, what each oscillator said, and how many agree. The entry is the day's high for a buy (slide 201) and the close for a sell, because the slides give no sell trigger. `write.csv()` saves the table in an output folder.

NEW IN STEP 5

```
signals <- data.frame(ticker = ticker, date = bars$Date[day], side = side,
                      entry = ifelse(side == "BUY", bars$High[day], bars$Close[day]),
                      rsi = word(rsi_sig[day]), macd = word(macd_sig[day]), stoch =
word(stoch_sig[day]),
                      agree = rowSums(cbind(rsi_sig, macd_sig, stoch_sig)[day, , drop = FALSE]
== ifelse(side == "BUY", 1, -1)))
dir.create("output", showWarnings = FALSE)
write.csv(signals, "output/signals.csv", row.names = FALSE, quote = FALSE)
```

6 Draw the chart

Price on top, the three oscillators in panels below. `points()` marks each signal day with a triangle, green up for a buy and red down for a sell, and the days where two or more oscillators agree are bigger and numbered with their row in `signals.csv`. `dev.off()` closes the picture file.

NEW IN STEP 6

```
png(file.path("output", paste0(ticker, ".png")), width = 1200, height = 1000)
layout(matrix(1:4), heights = c(3, 1, 1, 1))
par(mar = c(2, 4, 3, 1))
plot(bars$Date, bars$Close, type = "l", xlab = "", ylab = "Close", main = paste(ticker, "-
window oscillator signals"))
big <- signals$agree >= 2
points(signals$date, signals$entry, pch = ifelse(side == "BUY", 24, 25), bg = ifelse(side ==
"BUY", "darkgreen", "red"), cex = ifelse(big, 1.8, 0.7))
if (any(big)) text(signals$date[big], signals$entry[big], labels = which(big), pos = 3)
plot(bars$Date, rsi, type = "l", xlab = "", ylab = "RSI"); abline(h = 50, lty = 2)
matplot(bars$Date, macd, type = "l", lty = 1, xlab = "", ylab = "MACD"); abline(h = 0, lty = 2)
matplot(bars$Date, sto[, c("fastK", "fastD")], type = "l", lty = 1, xlab = "", ylab =
"Stochastic")
dev.off()
```

R PRINTED

```
null device
          1
```

7 Do it for every file

A `for` loop over `list.files()` runs everything once for each CSV in the data folder, adds each stock's signals to one table, and prints how many signal days it found. This is the finished script, exactly the file you were given.

> THE FINISHED SCRIPT
>
> `First Automation FIN1209 - Example Script.R`, 47 lines. The video types it.

WHAT R PRINTED

```
(R prints one line per stock)
```

R PRINTED

```
F : 25 signal days, 1 with two or more oscillators agreeing
KO : 27 signal days, 2 with two or more oscillators agreeing
```

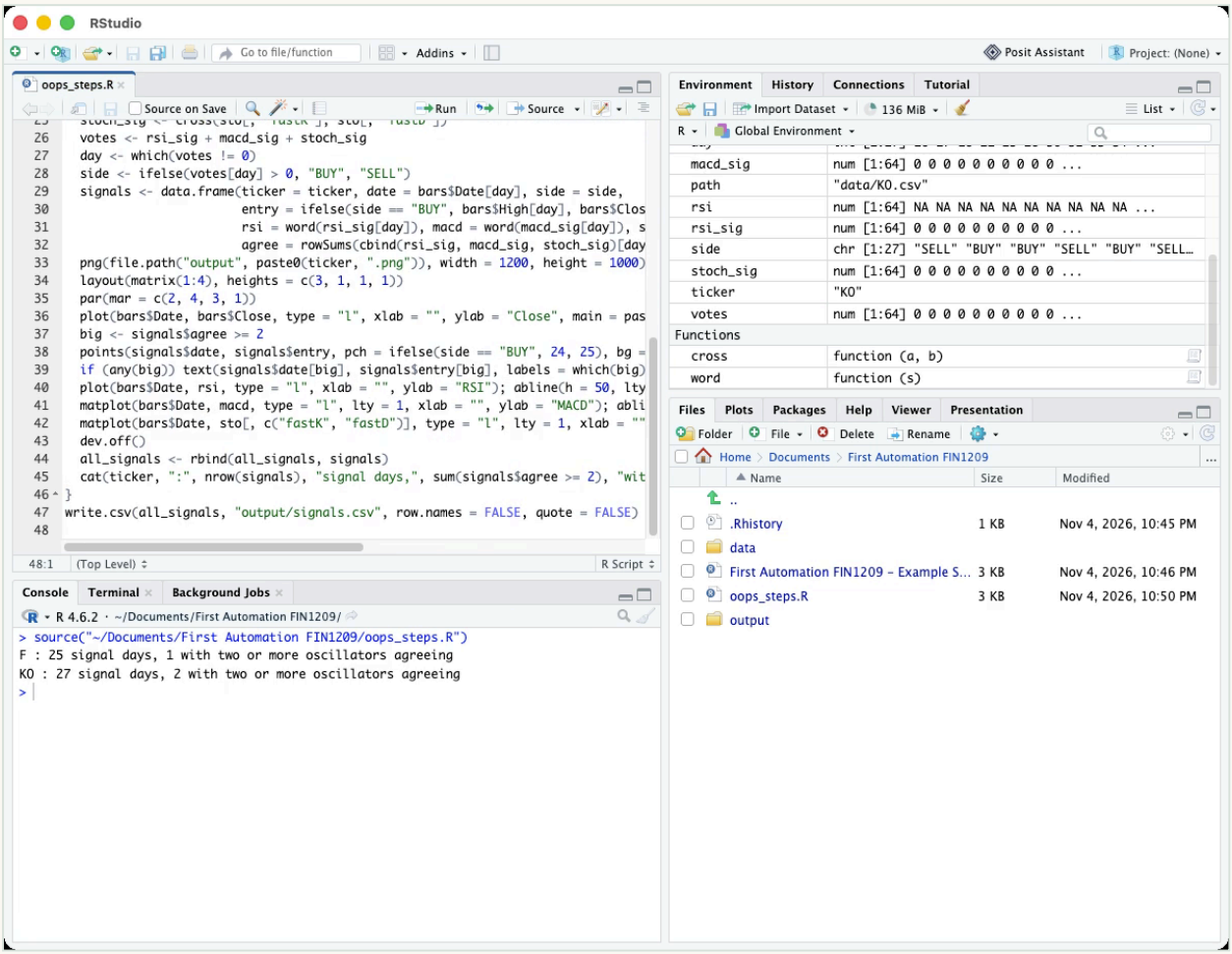
6. Run it and read the output

One answer per stock, a table, a chart. Then new data, and run it again.

1 Run it

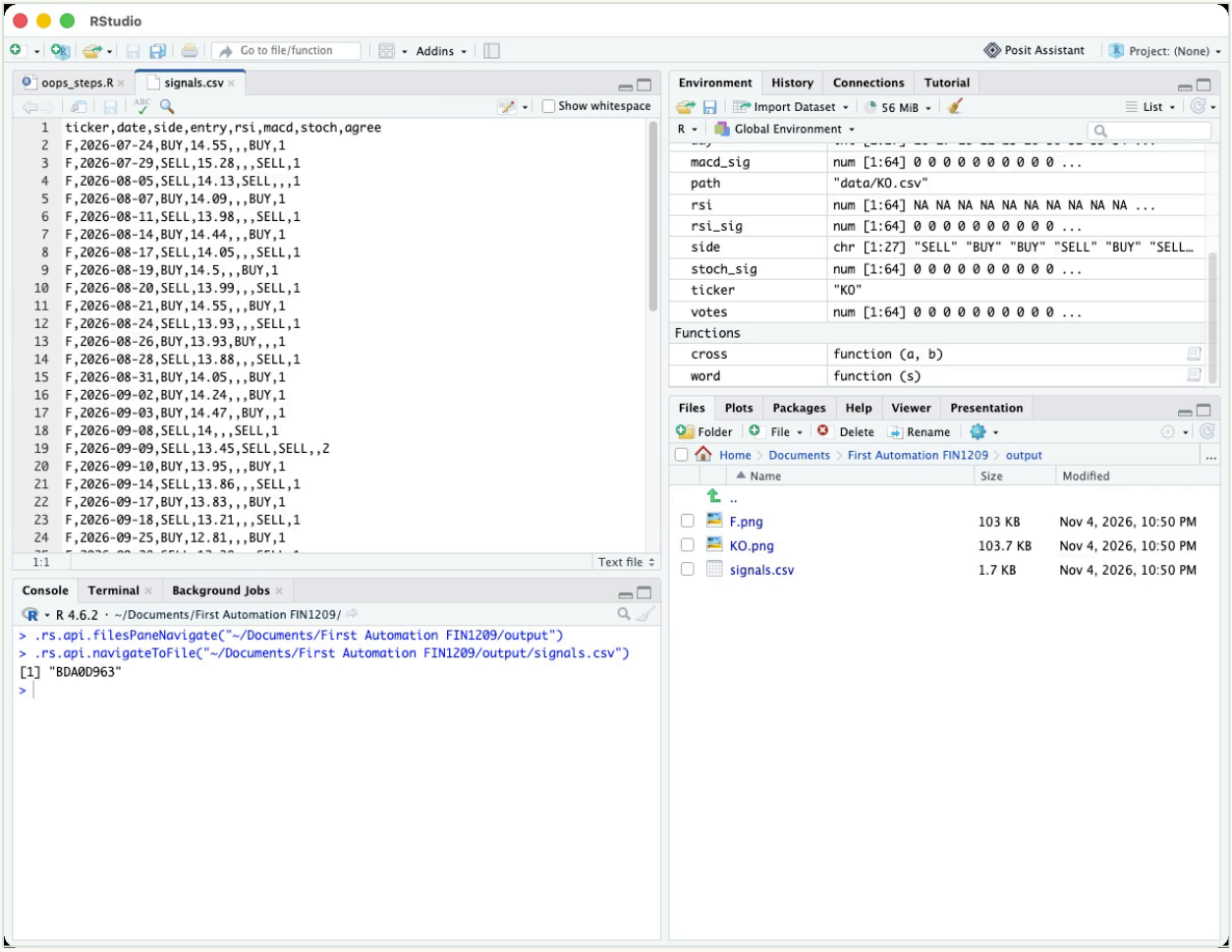
With the finished script open, click **Source**.

The Console prints one line per stock: how many signal days, and how many had two or more oscillators agreeing. First-run files: KO 27 signal days, F 25.



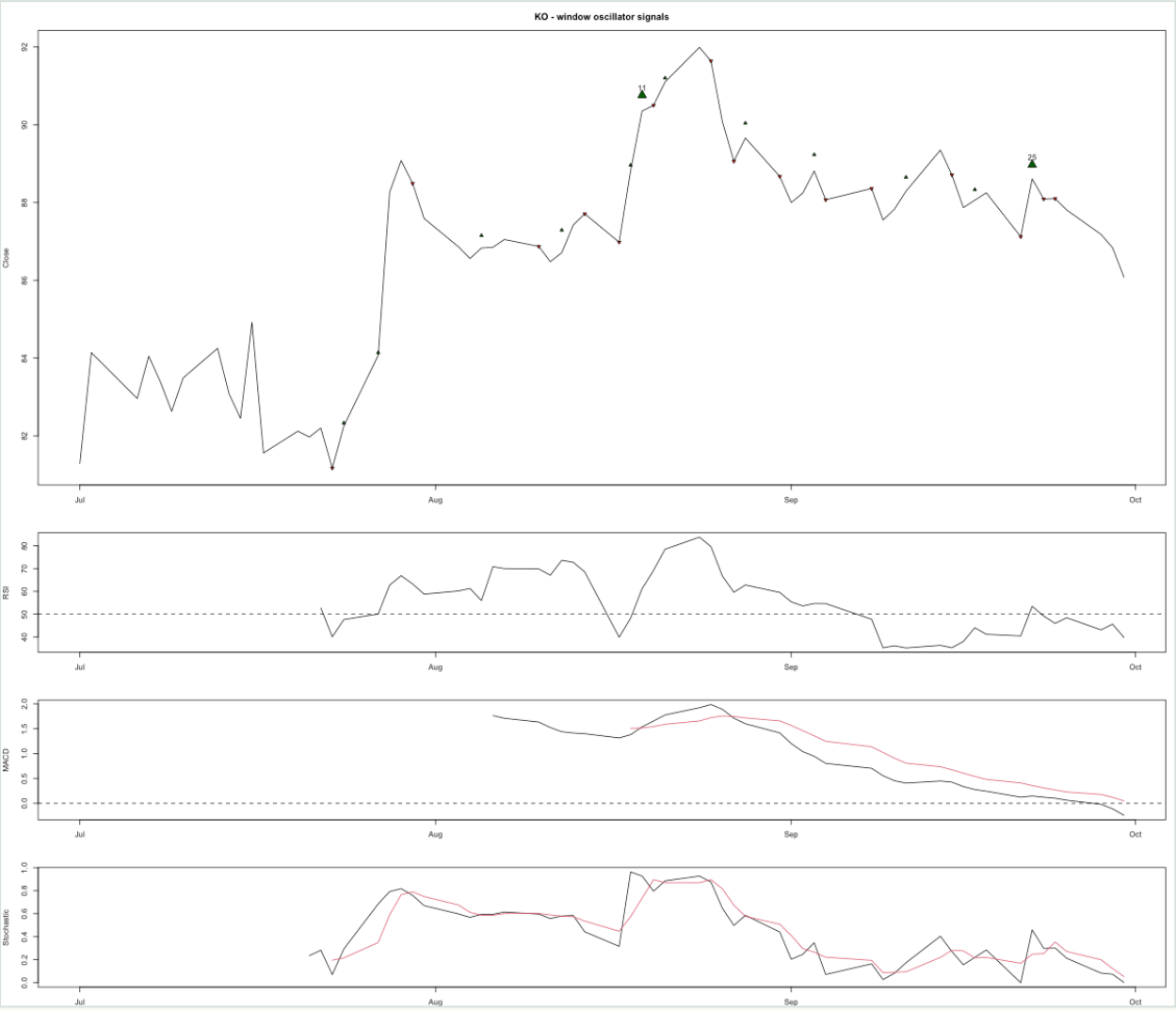
2 Read the table

Open `output/signals.csv`. Each row is a day on which an oscillator crossed: `side`, the `entry` price, what each oscillator said, and `agree`.



3 Read the chart

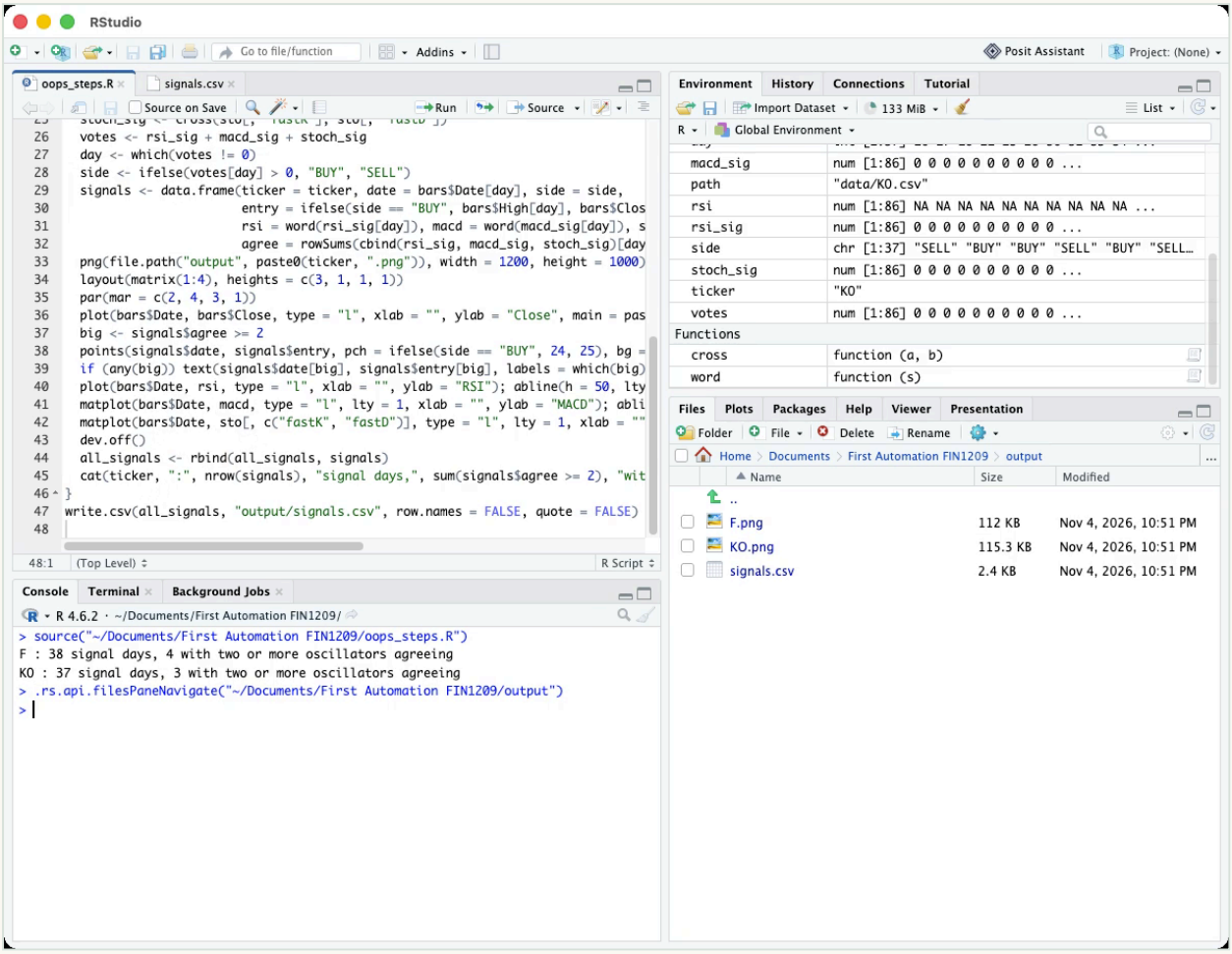
Open `output/KO.png`. Price is on top. Every triangle is a signal day, green up for a buy and red down for a sell; the bigger numbered ones are days two or more oscillators agreed. The three panels below are the oscillators. **The MACD panel is empty at the start:** the MACD needs about 34 days of history before it can give a signal, so the first weeks of this 64 day example show none. For your own stock pull about a year.



4 Add new data and run again

Download both files again with the later end date (Part 2), replace the ones in `data`, and click **Source** again. Nothing else changes.

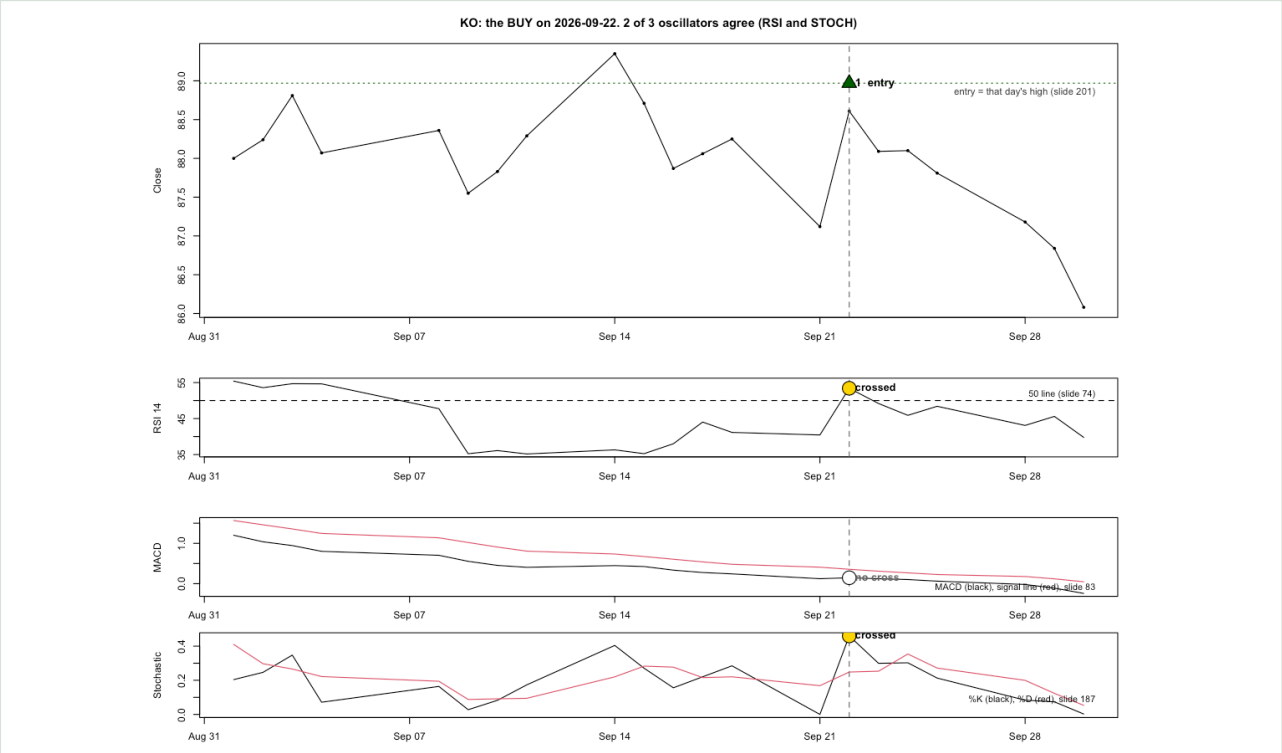
The counts go up: KO from 27 to 37 signal days, F from 25 to 38.



7. Why the signals are where they are

One real buy and one real sell from the example tables (KO and F), drawn for the weeks around the day.

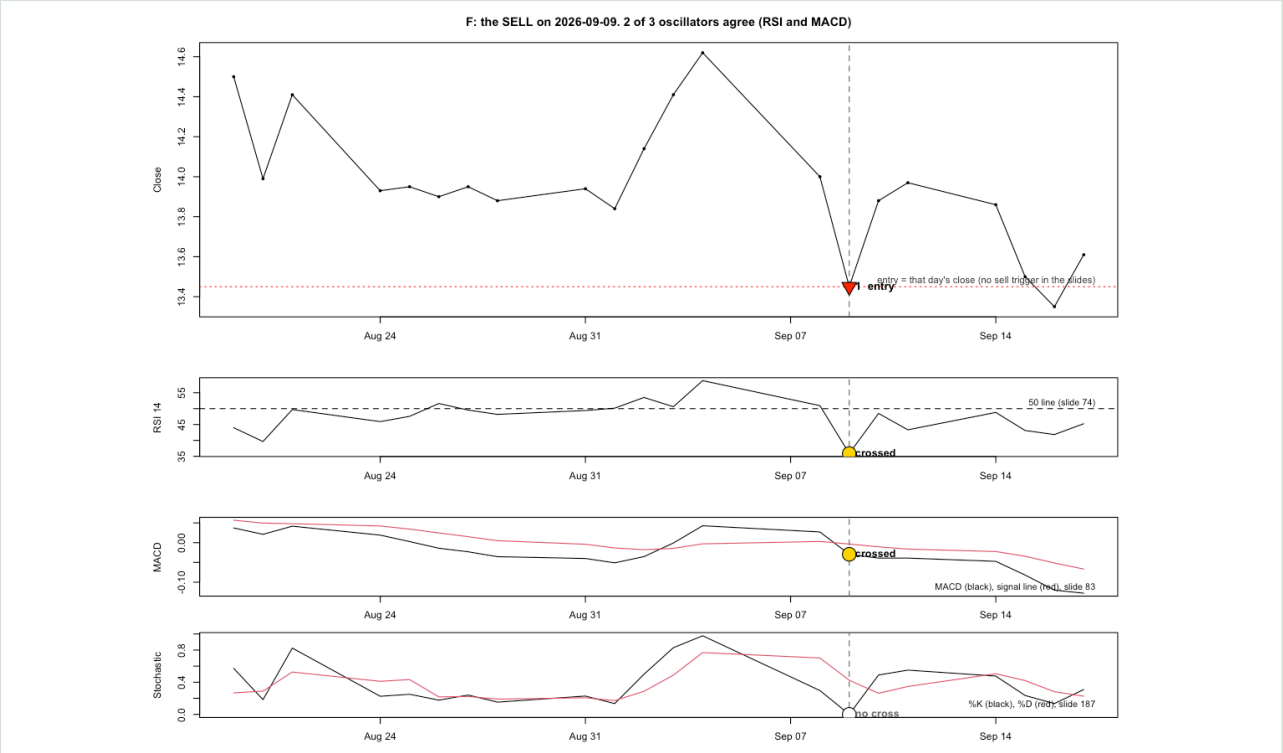
The script prints numbers only. This part is how to read them. **A buy.** On the day the RSI, the MACD or the stochastic crosses upward, that oscillator gives a buy signal. The price to act at is that day's high (slide 201). In the chart the gold circles mark the oscillators that crossed and the white ones the oscillators that did not.



KO, 2026-09-22: a BUY where RSI and STOCH agree, so two of three. Entry 88.97.

Drawn by build/r/explain_osc.R from the KO price file.

A sell is the mirror image: an oscillator crosses downward. The slides give no sell trigger, so the entry is the close of the day. That is our choice, not Lim's rule.



F, 2026-09-09: a SELL where RSI and MACD agree, so two of three. Entry 13.45.

Drawn by build/r/explain_osc.R from the KO price file.

> AGREEING IS NOT PROOF
>
> Three oscillators agreeing sounds like three witnesses. It is not. They are all built from the same closing prices, so they move together: on the example data the RSI and the stochastic have a correlation of 0.73 and the RSI and MACD 0.80. Lim calls this **multicollinearity** and warns that it creates a **false consensus** and a false sense of confidence (slide 78). His cure (slide 81) is oscillators built from different data, such as one that uses volume. A signal is only a statement of what a rule says that day, not a forecast.

8. The contract: what every script must do

Your script is marked by running it on files you have never seen, so everybody follows this exactly.

Folders and files

WHAT	RULE
Where it runs	From the folder that holds the script, which holds a folder called `data`.
Input	Every `.csv` in `data` is one stock; the ticker is the file name without `.csv`. At least 60 daily rows each, oldest first. Columns `Date, Open, High, Low, Close, Volume` exactly. `Date` may be `2026-07-02`, `2026-07-02 16:00:00` or `7/2/2026 16:00:00` (Google Sheets).
Output	`output/signals.csv`, every stock together, and one `output/<ticker>.png` per file: price on top, your oscillators below, signal days marked.
Packages	Base R and `TTR`. Nothing else, and no `install.packages` inside the script.
Time	Under two minutes for three files of a year each.

The columns of output/signals.csv, in this order

COLUMN	MEANING	EXAMPLE
`ticker`	The file name without `.csv`	`KO`
`date`	The signal day, `YYYY-MM-DD`	`2026-04-08`
`side`	`BUY` or `SELL`: the side most oscillators gave	`BUY`
`entry`	A buy: the day's high. A sell: the day's close	`77.37`
`rsi, macd, stoch`	One column for each oscillator you use, from `rsi`, `macd`, `stoch`. `BUY`, `SELL` or empty. **Use at least two.** The columns you use tell the marker your pair.	`BUY`
`agree`	How many of your oscillators gave the `side`	`2`

One row per day on which at least one oscillator crosses. Rows go ticker by ticker, then by date. The chart number of a day is its row number in the file.

9. Your task

Your own script, on your own stock, and a flowchart of it.

Choose **your own stock** (a US stock, downloaded as in Part 2) and **your own pair** of the three oscillators: RSI and MACD, RSI and stochastic, or MACD and stochastic. Write your own R script that meets the contract in Part 8 for your pair, and draw a **flowchart** of how it works.

> DUE DATE
> _____ (to be filled in: after the Saturday 7 November class)

Name your files exactly

FILE	NAME
Your script	`FirstAutomation_Surname_Firstname.R`
Flowchart, PDF (preferred)	`FirstAutomation_Surname_Firstname_Flowchart.pdf`
Or the .bpmn file	`FirstAutomation_Surname_Firstname_Flowchart.bpmn`
Or a Markdown file	`FirstAutomation_Surname_Firstname_Flowchart.md`

Two files, nothing else. No price files or output: they are rebuilt from your script.

Before you submit, check

1. Put your script in a clean folder with a `data` folder of the example files and run it, with no changes.
2. `output/signals.csv` has `ticker, date, side, entry`, your oscillator columns and `agree`, and a `.png` per file is in `output`.
3. Delete `output` and run again. It must come back.
4. Open your flowchart next to your script. Every shape has lines behind it.

> THE MARKER RUNS YOUR SCRIPT ON FILES YOU HAVE NOT SEEN
> Real stocks, a different period, in the layout of Part 8. Some files write dates as `2026-07-02`, some as `7/2/2026 16:00:00`. A script that only works on the example files will not pass.

10. Using AI

Allowed, with one condition.

You may use AI to help write your script. The condition is that **you can properly visualise and explain it in your flowchart**. The flowchart is where you show that you understand the script, so:

- The flowchart must match **the script you actually submit**, not an earlier version and not what the AI described.
- Each task must say, in your own words, what that part of the code does, and where the rule has a number, which slide it comes from.
- If you cannot explain a part of the script, do not submit it. Ask for a simpler version you can explain, or go back to the example.

A script that runs perfectly with a flowchart that does not match it loses the flowchart marks. A modest script with a flowchart that matches it and explains every step does well.

11. The marks

100 points. 60 are checked against the record by running your script, and 40 are judged from your flowchart.

Checked against the record

WHAT	MARKS	HOW
The script runs	10	On a folder of files you have never seen, with no changes and no installs, in under 60 seconds.
It writes the files the contract names	10	`output/signals.csv` with the required columns (5), and one `output/<ticker>.png` for every input file (5).
It finds the right signals	20	The ticker, date and side of every signal match the record, with none missing and none extra.
It gets the prices right	10	Entry and stoploss match the record, to the cent.
The chart shows them	10	Each chart is a real chart with every signal marked and numbered.
Subtotal	**60**	

Right or wrong. No rubric is needed because the record decides.

Judged, with the course rubric on the next pages

WHAT	MARKS
The flowchart is correct BPMN	10
The flowchart matches your script	15
The flowchart explains each step	15
Subtotal	**40**

NOT MARKED

How your code looks. Neat or messy, short or long, with comments or without: not marked. Only whether it works and whether the flowchart explains it.

YOUR FLOWCHART IS MARKED ON ITS OWN PAGES, AND THE SIGNALS ARE NOT A FORECAST

The script reports what a rule says at a point in the past. Nothing here is marked on whether a signal would have made money.

How a written answer is marked

For the 40 points that a record cannot check. The same rubric marks every written answer in this course.

> READ THIS FIRST
>
> **We are marking whether your answer is grounded and whether it follows, not how long it is.** A long answer that wanders earns less than two precise sentences.

This rubric is for written answers that have no single correct response. Anything that can be checked against the record, a price, a date, a count, is simply right or wrong, and no rubric is needed for it.

What earns the marks

- **Grounded.** Your answer points at something specific and checkable on your own chart: a price, a date, a shape. Not "the market looks bullish", but "the trough on this date, at this price, is higher than the one before it".
- **Connected.** Your conclusion follows from what you pointed at. It is possible to name three real things and then conclude something that has nothing to do with them.

> HONEST: NAMED, NEVER MARKED
>
> A strong answer also names what argues against it. This is what the best answers do, and it is worth learning to do. **No marks are lost for leaving it out.**

The levels

LEVEL	SHARE OF THE MARKS	WHAT IT LOOKS LIKE
Full	100 percent	Grounded and connected.
Partial	75 percent	One of the two is weak: the evidence is vague, or the conclusion does not quite follow from what was named.
Minimal	50 percent	There is a reason, but it is general rather than taken from your own chart.
None	0 percent	A conclusion with no reasoning, or nothing written.

Percentages rather than marks, so the same rubric marks a question worth four points or forty.

The written answers on this sheet, level by level

ANSWER	OUT OF	FULL, 100%	PARTIAL, 75%	MINIMAL, 50%	NONE, 0%
The flowchart is correct BPMN	10	10	7.5	5	0
The flowchart matches your script	15	15	11.25	7.5	0
The flowchart explains each step	15	15	11.25	7.5	0

Every figure in this table is the level's percentage of what the answer is worth, so it cannot disagree with the levels above.

What the rubric looks for in each of them

ANSWER	GROUNDED HERE MEANS	CONNECTED HERE MEANS
The flowchart is correct BPMN	Points at something checkable in your diagram: a start and an end event, a labelled `yes` and `no` leaving every gateway, an arrow back for every loop.	The shapes are used for what they mean: a diamond is a question, a rectangle is an action, and no path stops without an end event.
The flowchart matches your script	Points at the line of your script each shape stands for, and at every loop and decision in the code having a shape.	Following the arrows reproduces what running the script does, in the same order, with no shape for code that is not there.
The flowchart explains each step	Each task says what its lines do, with the actual column, price or file the code uses, and names the slide where the rule has a price.	The explanation tells the reader why that step is there, not only what it is called.

One question, answered at every level

Here is task 5 of the example flowchart explained five ways, one at each level, and two at Partial because there are two ways to be partial. They are written on the example, not on yours.

LEVEL	THE ANSWER	WHY IT SITS THERE
Full, 100%	Task 5, Add the votes and keep the signal days: it adds the three crossing signals, so a day with one buy and one sell cancels to zero and is skipped. Days with a total above zero are BUYs with entry = High, as slide 201 says, and `agree` counts the oscillators that gave that side.	Names what the code adds, a real consequence (cancelling), the exact price rule and the slide.
Partial, 75%	Task 5, Add the votes and keep the signal days: it adds the signals and keeps the days with a signal, using the slide 201 entry price.	Right idea and a slide, but it never says what is added or what `agree` counts, so it cannot be checked.
Partial, 75%	Task 5, Add the votes and keep the signal days: it adds the three signals and keeps the days that are not zero, because three oscillators agreeing means price will rise.	Checkable action, but the reason does not follow: the code does not predict anything.
Minimal, 50%	Task 5, Add the votes and keep the signal days: this is where the program decides which days matter.	A reason, but general: it could describe any task.
None, 0%	Task 5, Add the votes and keep the signal days: votes.	A label with no reasoning.

> WHAT THE BEST ANSWERS ADD
>
> A strong flowchart also says where its script is not Lim's rule: "the slides give no sell trigger, so this script uses the close." It is never marked, and it is the habit that makes a rule you automated one you can trust.